

Road Network Planning Combined with Sand Table Map Case Study

Road Network Planning Combined with Sand Table Map Case Study

1. Course Content
2. Sand Table Lane Network Analysis
3. Start the road network planning and navigation node
4. Test Sandbox Road Network Navigation Case
 - 4.1 Point-to-Point Navigation Between Road Networks
 - 4.2 Arbitrary Pose Road Network Planning and Navigation
5. Source Code Analysis
 - 3.1 route_bridge.py Analysis
 - 3.1.1 Waypoint Navigation
 - 3.1.2 Arbitrary Pose Navigation
 - 3.2 Analysis of conver_speed.py
6. Navigation Parameter Debugging
 - 6.1 Navigation Controller Debugging
 - 6.2 Obstacle Avoidance and Cost Map Debugging
7. Notes

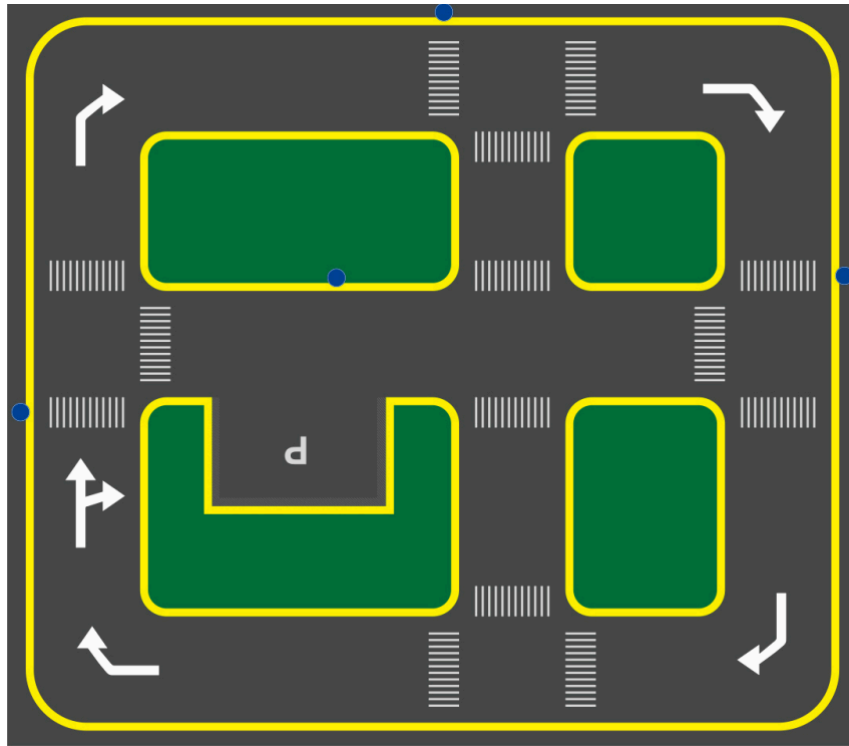
1. Course Content

Important

- 1. Using road network planning navigation, combined with a functional sand table map to build an urban vehicle road network, allowing the vehicle's navigation system to follow lane traffic rules and plan routes that conform to the road network when navigating in the sand table.
- 2. Subsequent courses will include AI large-scale models combined with sand table autonomous driving and road network planning navigation case studies. This section explains how to build road network planning navigation in the sand table.

2. Sand Table Lane Network Analysis

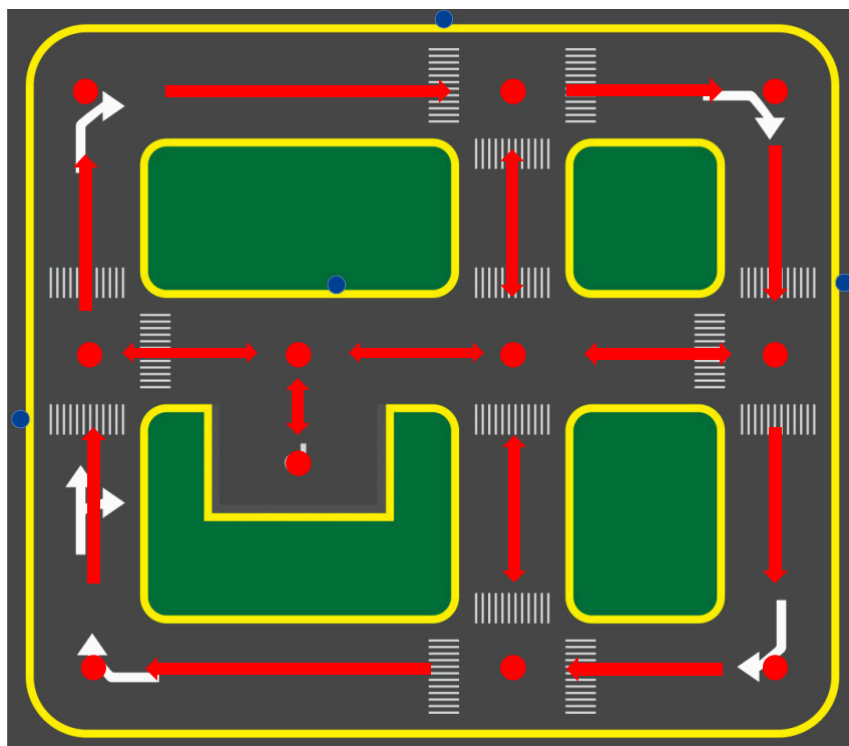
- Here we can see that the outermost ring lanes are all **one-way lanes**, allowing only clockwise one-way travel, while the middle lanes are two-way lanes. Therefore, we can use all corners and intersections as waypoints to connect edges and build the entire lane topology network.



- The lane traffic rule network is shown in the following figure.

 Tip

The following figure shows an example lane traffic network. Users can also add more waypoints or modify the lane network direction according to their actual needs.



3. Start the road network planning and navigation node

- (Host side) Chassis radar driver

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```

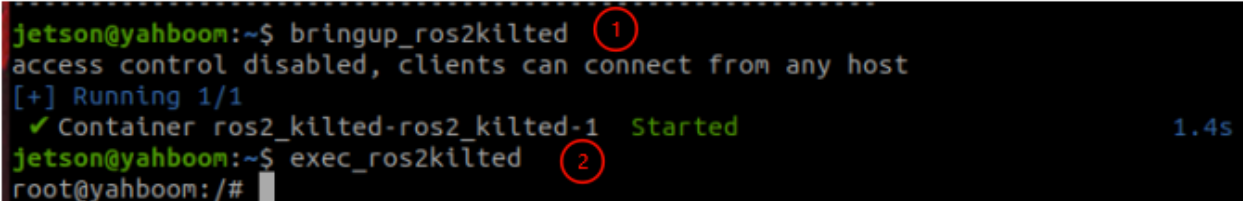
- Starting the road network visualization on the Docker container

Open Docker and run the following command in the terminal:

```
bringup_ros2kilted
```

Enter the Docker container and run the following command in the terminal:

```
exec_ros2kilted
```

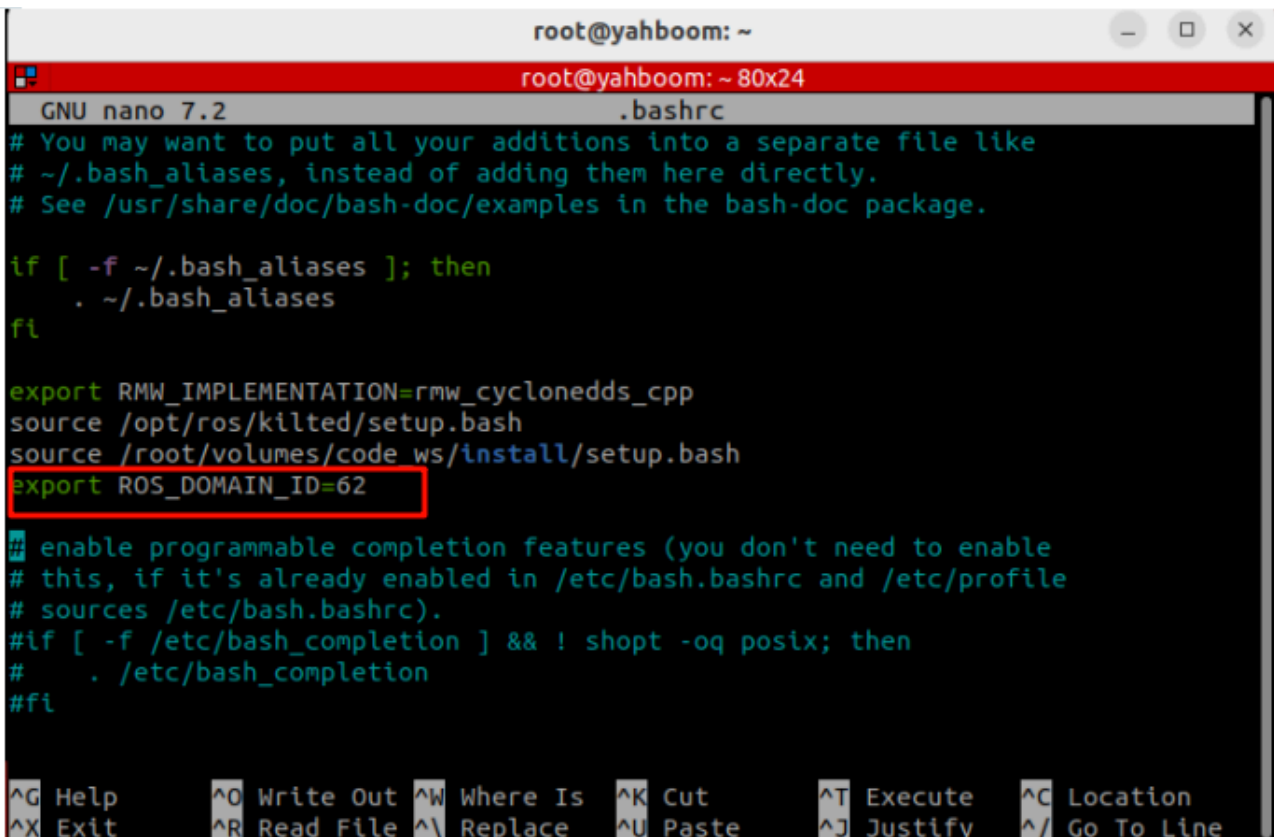


```
jetson@yahboom:~$ bringup_ros2kilted
access control disabled, clients can connect from any host
[+] Running 1/1
✓ Container ros2_kilted-ros2_kilted-1 Started 1.4s
jetson@yahboom:~$ exec_ros2kilted
root@yahboom:/#
```

Modify `do_main_id` to match the host machine:

```
nano .bashrc
# After modification, source the current environment
source .bashrc
```

After making the changes, press Ctrl+x, then enter y and press Enter to save and exit.



```
root@yahboom: ~
GNU nano 7.2 .bashrc
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
source /opt/ros/kilted/setup.bash
source /root/volumes/code_ws/install/setup.bash
export ROS_DOMAIN_ID=62

## enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
#if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
#    . /etc/bash_completion
#fi

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

Start the road network planning and navigation node (in the Docker terminal):

```
ros2 launch road_net_route roadnet_nav2.launch.py map:=map.yaml pose_map:=map
roadnet_file:=/root/map/map.geojson
```

- Enabling a visual interface in rdk on the virtual machine

```
ros2 launch road_net_route rviz_only.launch.py
```

- (Host machine) Start **cartograph** for fast localization

```
#orin
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/jetson/map/map.pbstream
#rdk
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/sunrise/map/map.pbstream
```

Tip

Optional Startup Parameter Description

map The name of the raster map file to be loaded, defaulting to `map.yaml`

load_state_filename The pose map file to be loaded. This must be passed when using cartograph for pure localization in navigation.

roadnet_file The road network description file, requiring a complete file path.

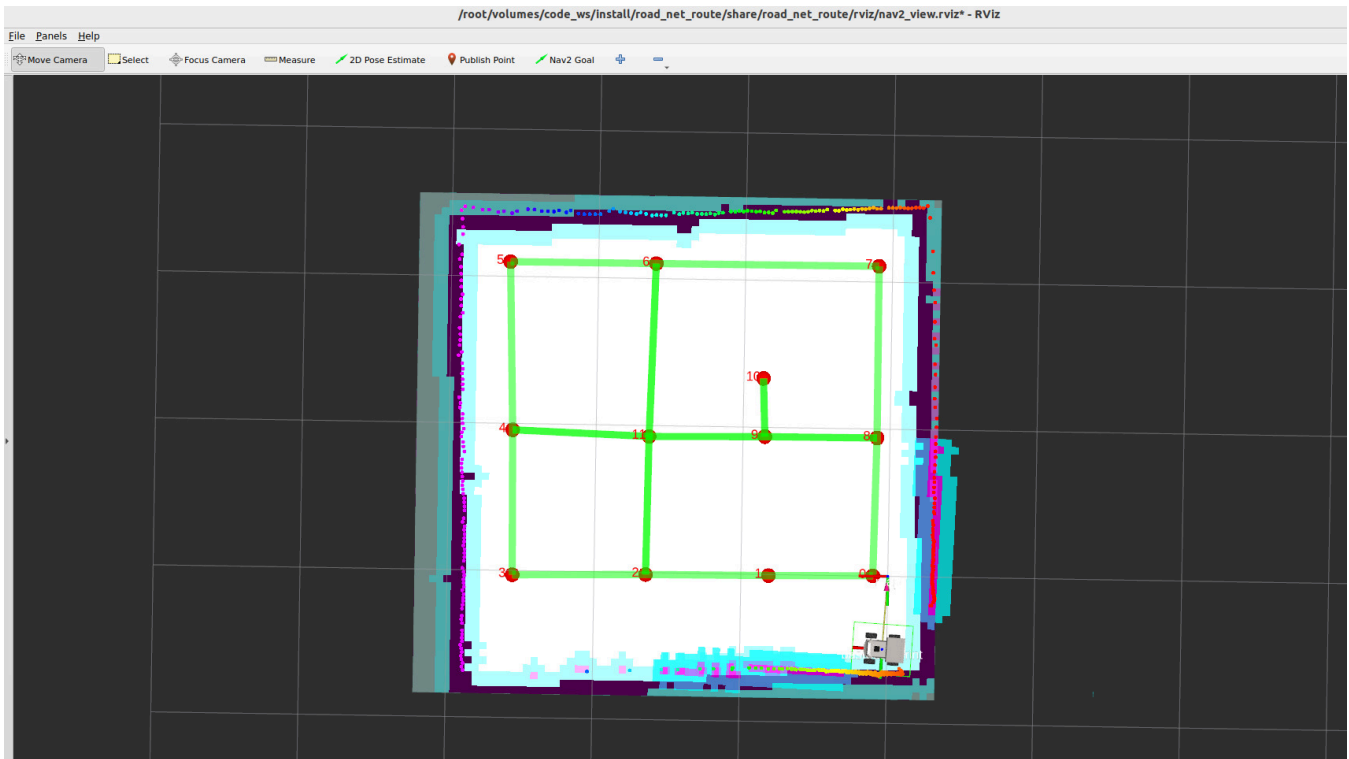
params_file The navigation parameter file, defaulting to

`os.path.join(get_package_share_directory('road_net_route'), 'config', 'nav2_kilted.yaml')`

localization_mode The localization mode, defaulting to cartograph.

Parameter name	Input	Final actual path
<code>map</code>	<code>map.yaml</code>	<code>/root/map/map.yaml</code>
<code>pose_map</code>	<code>map</code>	<code>/root/map/map</code>
<code>roadnet_file</code>	Absolute path	<code>/root/map/map.geojson</code>

- Place the car on the map and use the **2D Pose Estimate** tool to give the car its initial pose on the map we created. This approximate position on the map allows for quick and accurate location of the vehicle's global position.



4. Test Sandbox Road Network Navigation Case

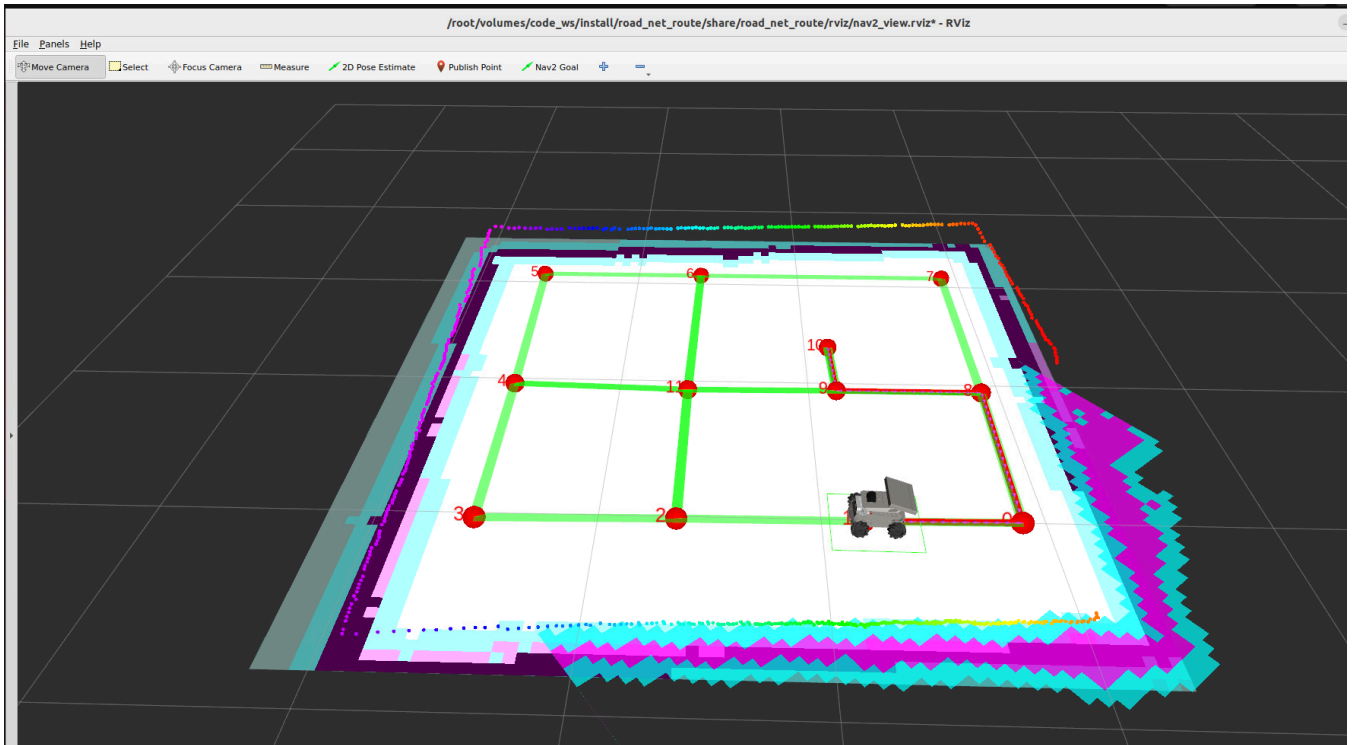
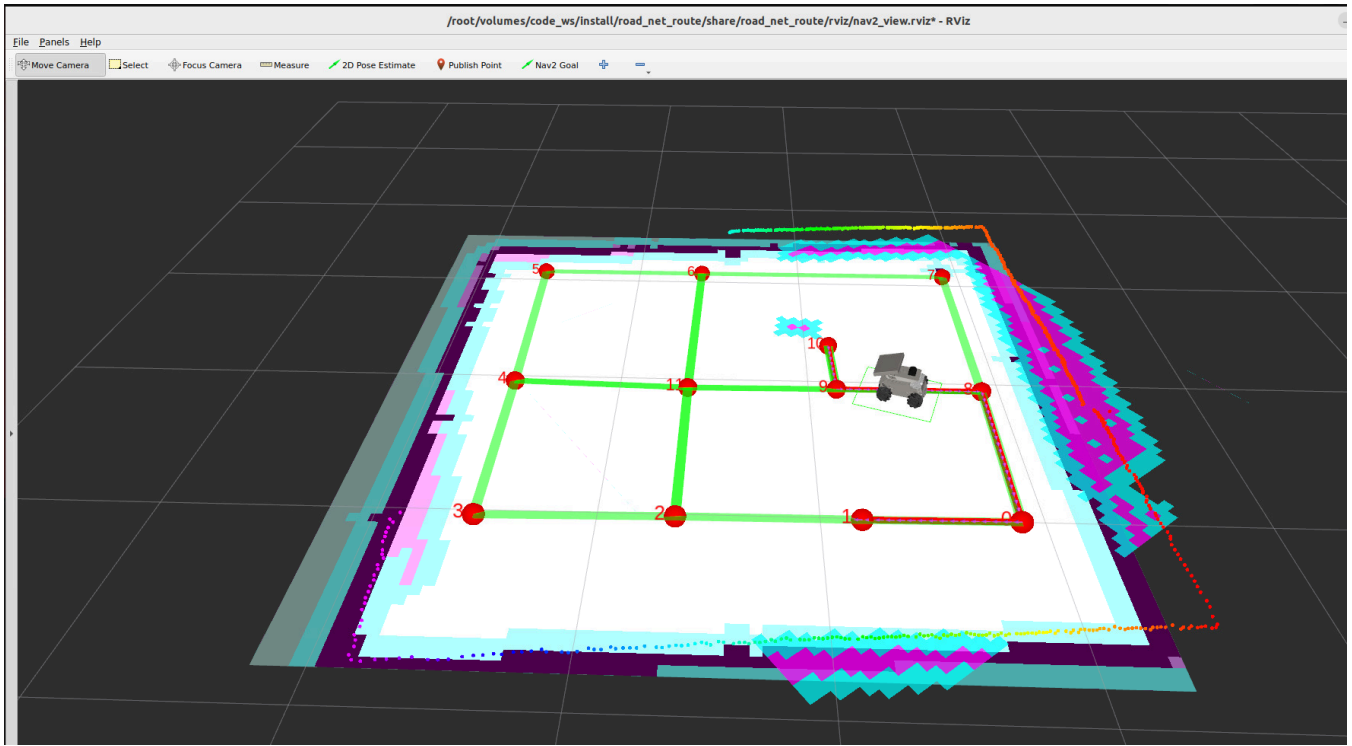
4.1 Point-to-Point Navigation Between Road Networks

- Here we demonstrate using the navigation from point 10 in the road network to point 1 in the road network. Publish a topic in the terminal:

Note

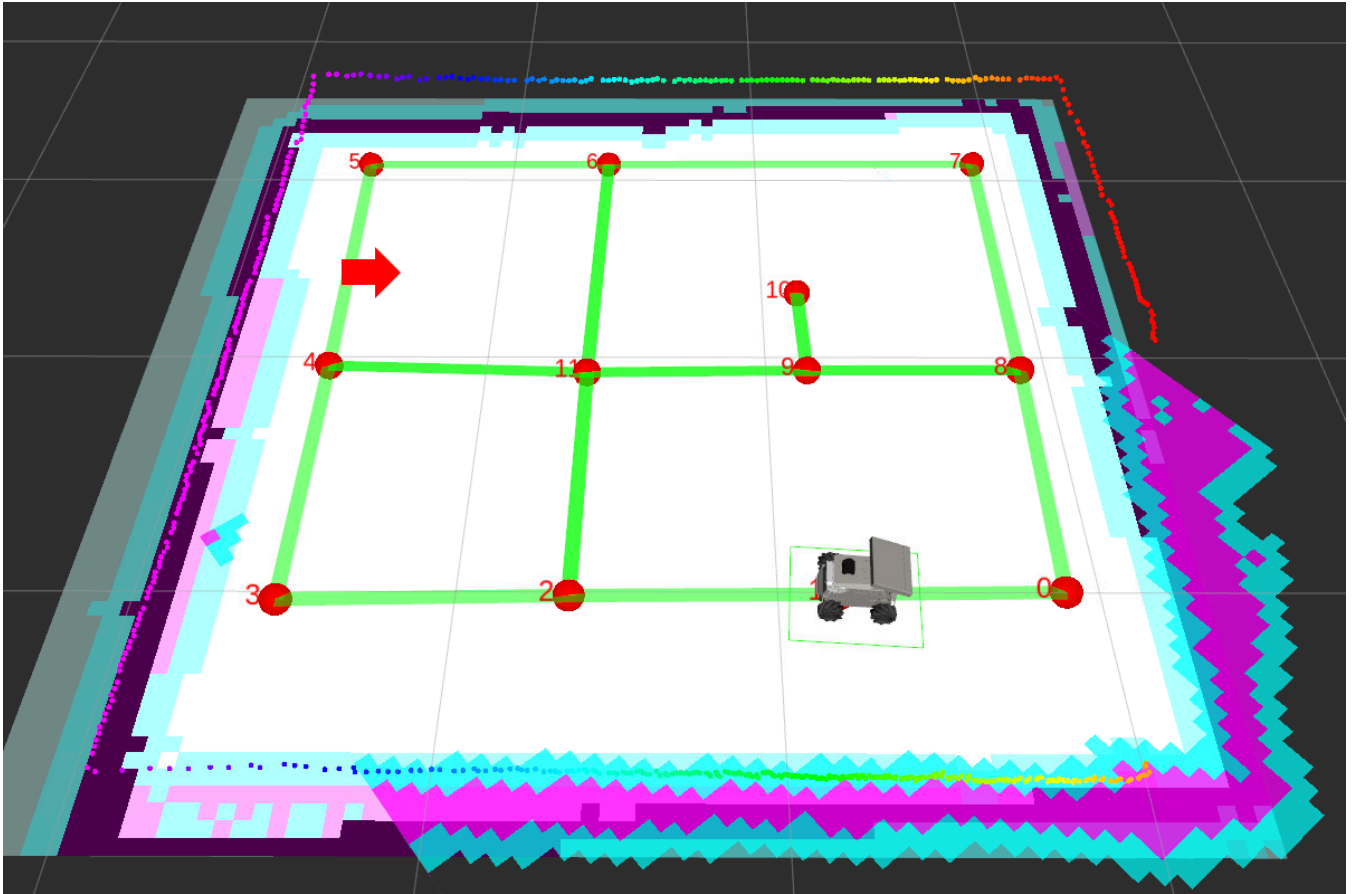
If the starting position is not a point or edge in the road network, when performing road network navigation, the robot will automatically first enter the nearest road network edge and then move along the road network.

```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [10, 1]}"
```

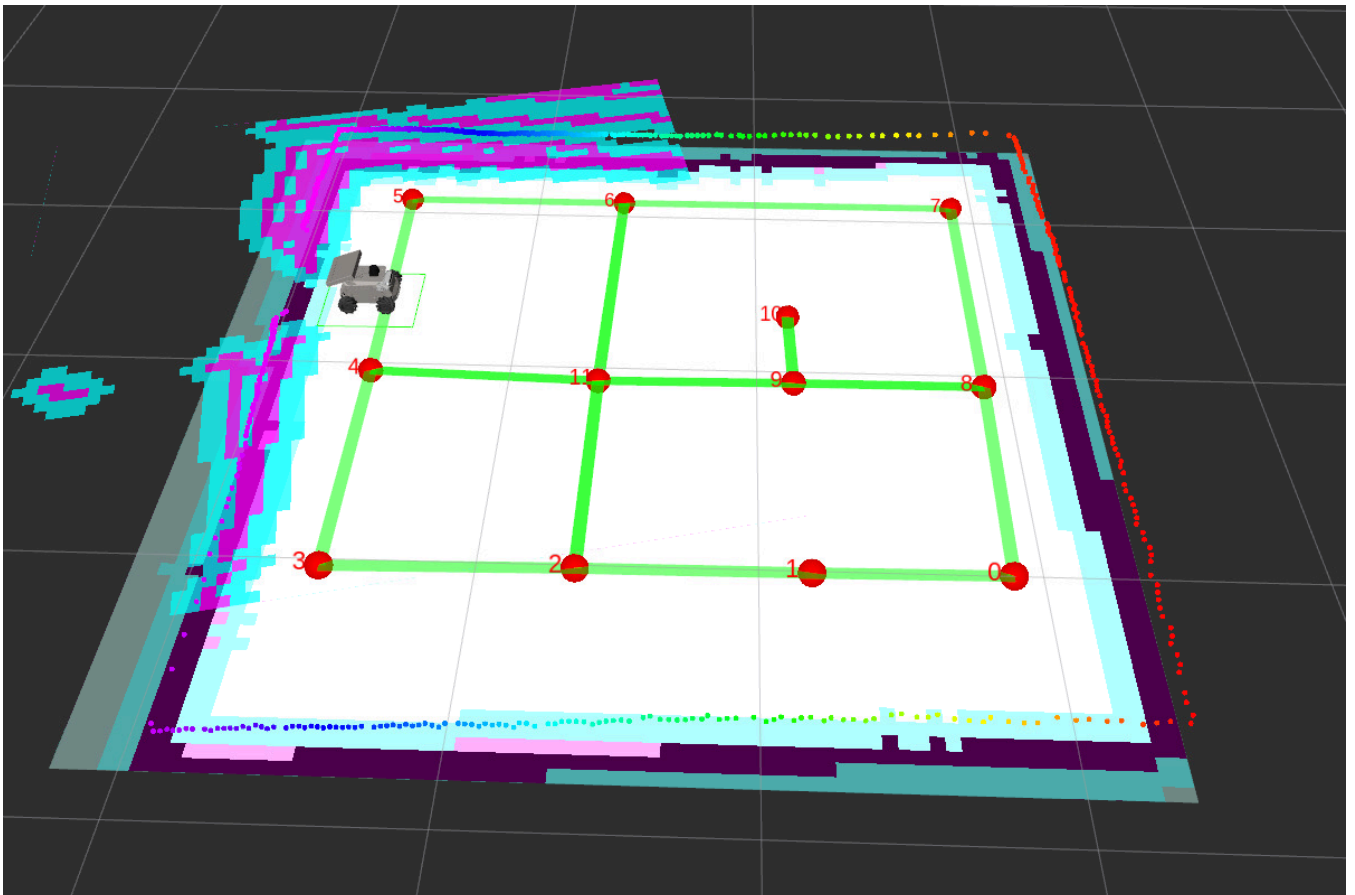


4.2 Arbitrary Pose Road Network Planning and Navigation

- Here we demonstrate using the arrow direction in the image as an example:



- First, control the robot to move to this pose (keyboard control, gamepad control, RViz 2D Goalt Pose navigation).

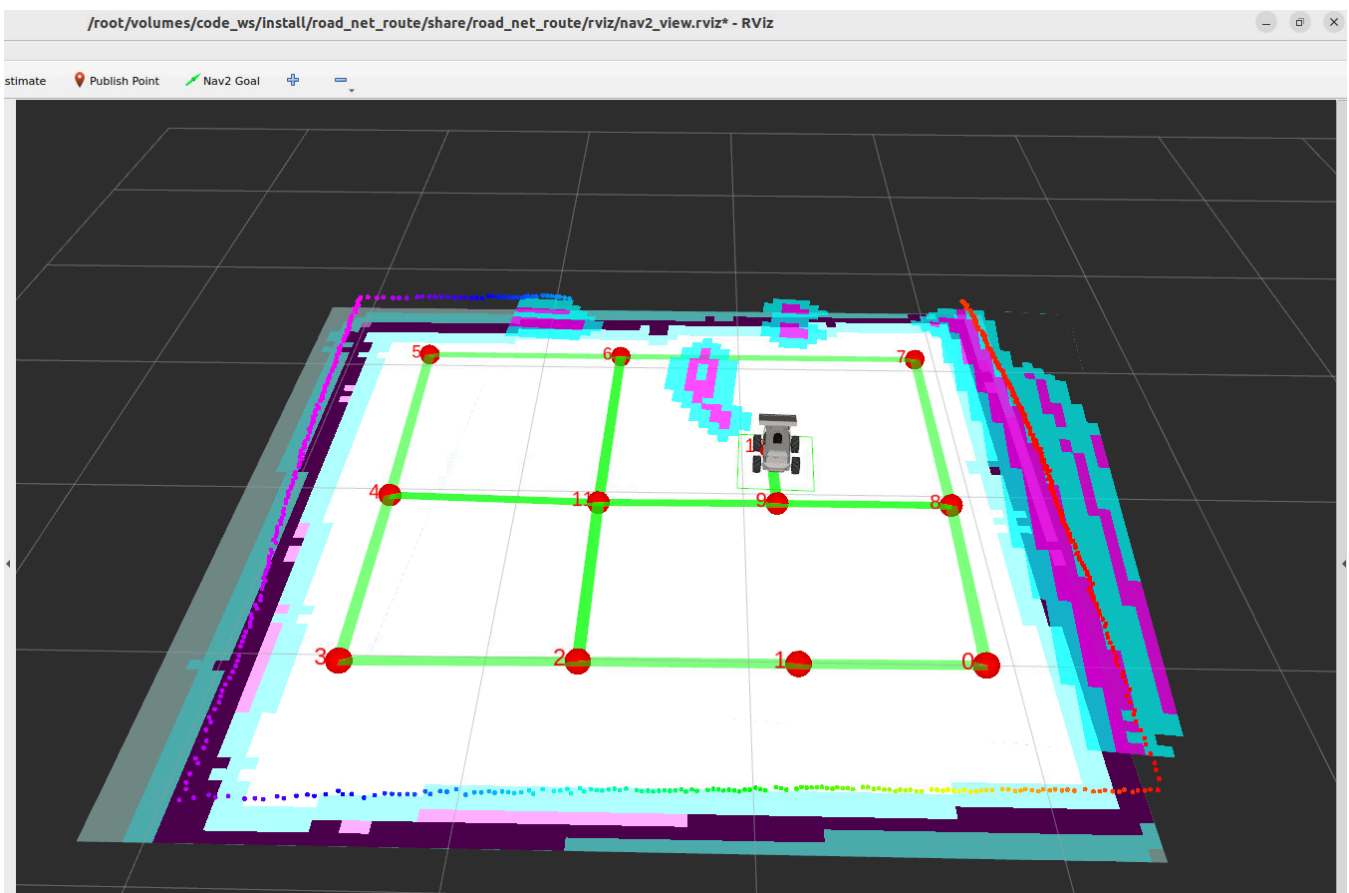


- Use the following command to view the robot's current coordinates on the map:

```
ros2 run tf2_ros tf2_echo map base_footprint
```

```
jetson@yahboom: ~ 68x18
- Rotation: in RPY (degree) [0.000, -0.000, 179.949]
- Matrix:
-1.000 -0.001  0.000  2.648
 0.001 -1.000  0.000 -1.388
 0.000  0.000  1.000  0.000
 0.000  0.000  0.000  1.000
At time 1763622288.978000000
- Translation: [2.648, -1.388, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, 1.000, 0.000]
- Rotation: in RPY (radian) [0.000, -0.000, 3.141]
- Rotation: in RPY (degree) [0.000, -0.000, 179.985]
- Matrix:
-1.000 -0.000  0.000  2.648
 0.000 -1.000  0.000 -1.388
 0.000  0.000  1.000  0.000
 0.000  0.000  0.000  1.000
^C[INFO] [1763622290.212653804] [rclcpp]: signal_handler(signum=2)
jetson@yahboom: ~$
```

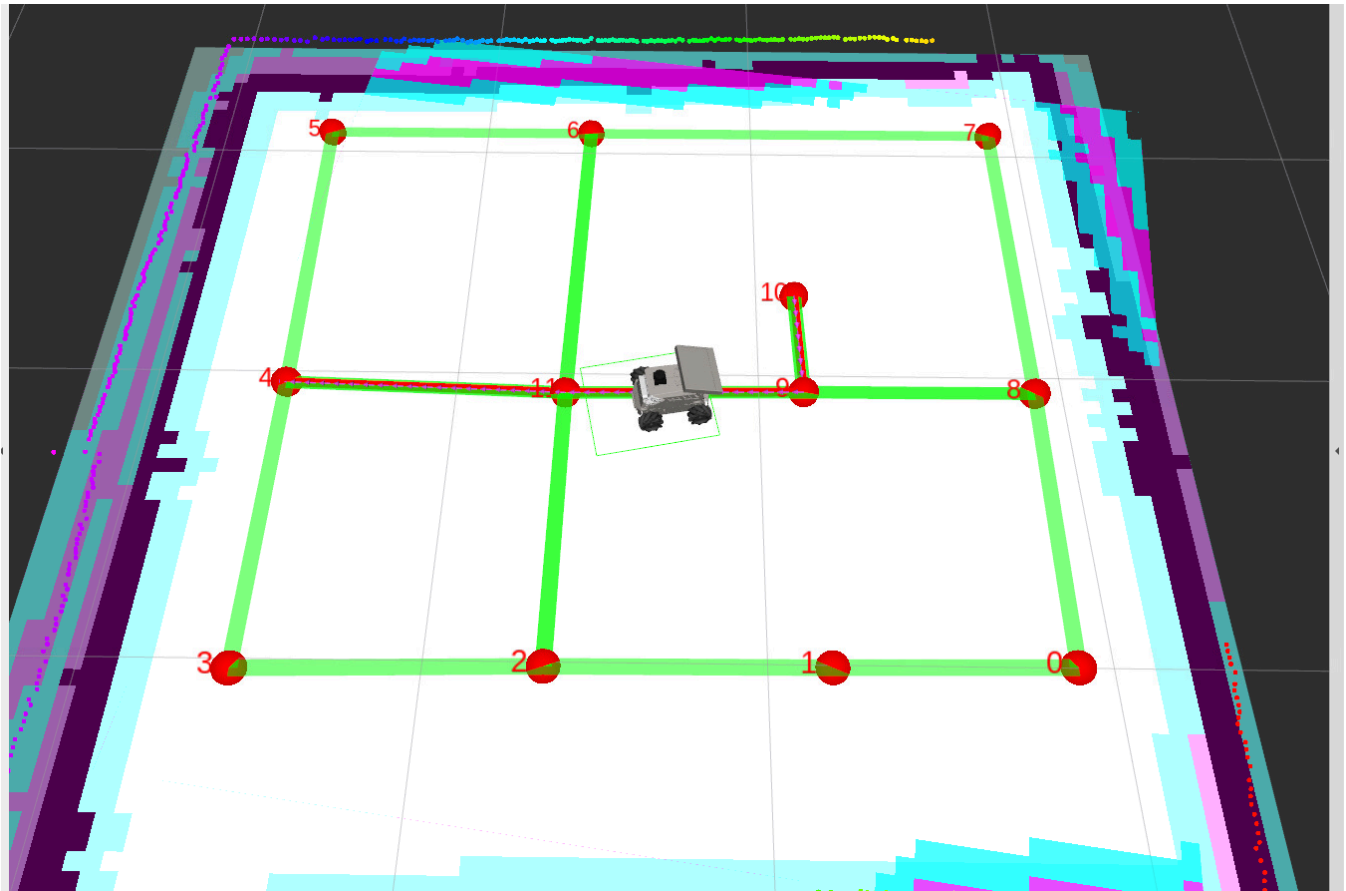
- Then, have the vehicle's infotainment system return to Waypoint 10 (this example uses Waypoint 10; it can be located anywhere on the map or within the road network).



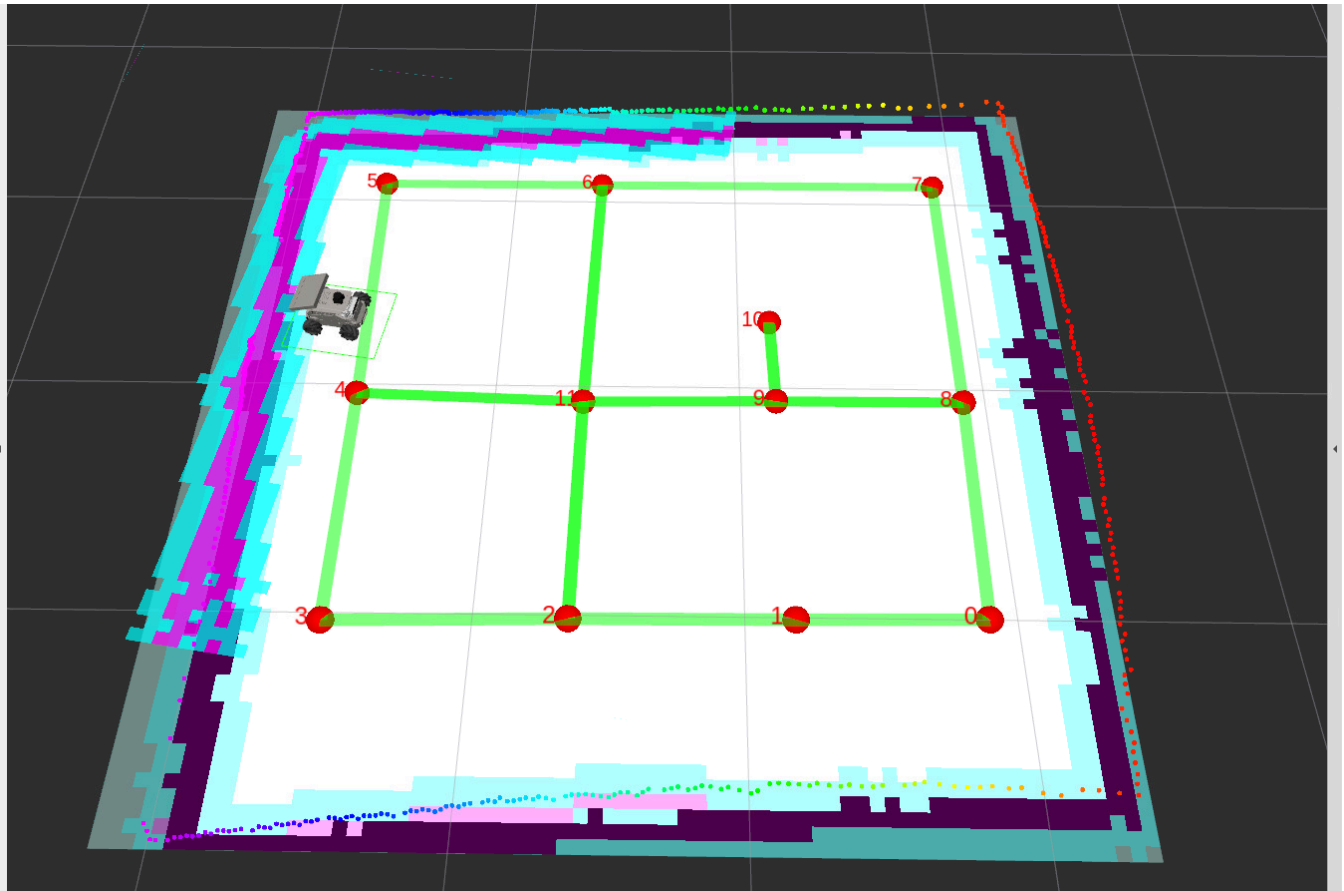
- Publish the target pose topic on the terminal.


```
ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  header: {frame_id: 'map'},
  pose: {
    position: {x: 2.648, y: -1.388, z: 0.0},
    orientation: {x: 0.0, y: 0.0, z: 1.000, w: 0.000}
  }
}"
```

- The vehicle will first follow the net to the nearest waypoint to the target position.



- Then adjust to the target pose.



5. Source Code Analysis

- Implementation program source code path:

```
~/ros2_kilted/volumes/code_ws/src/road_net_route/road_net_route/route_bridge.py
```

```
~/ros2_kilted/volumes/code_ws/src/road_net_route/road_net_route/conver_speed.py
```

3.1 route_bridge.py Analysis

3.1.1 Waypoint Navigation

- The initialization function of the `Route_Bridge` class creates a `road_net_nav_id` topic subscriber, receiving `Int16MultiArray` data (format `[start_id, goal_id]`) from the `road_net_nav_id` topic. It directly calls `getAndTrackRoute` to initiate road network navigation:
- Input validation: If the data length is not 2, an error is reported; if it is `[-1, -1]`, navigation is canceled.
- Navigation execution: Calling `self.navigators.getAndTrackRoute(start_id, goal_id)` returns a navigation task handle.
- Feedback Processing: Loop through navigation feedback (`feedback`), listen for node switching (`last_node_id`/`next_node_id`) and rerouting (`rerouted`) events. If rerouting occurs, call `followPath` to update the local path.
- Result Processing: Output logs based on `TaskResult` (success/cancellation/failure), and publish the results via `/action_feedback`.

```
self.road_net_nav_ID_sub= self.create_subscription(Int16MultiArray, "road_net_nav_id",
self.id_nav_callback, 10)
```

```
def id_nav_callback(self, msg: Int16MultiArray):
    ''' callback function for navigation route messages using IDs '''
    if len(msg.data) != 2:
        self.get_logger().error(Fore.RED+"Invalid ID navigation message
received."+Fore.RESET)
        return
    if msg.data[0]==-1 and msg.data[1]==-1 :
        self.navigator.cancelTask()
        self.get_logger().info(Fore.YELLOW+"navigation task canceled via
message."+Fore.RESET)
        return
    route_tracking_task = self.navigator.getAndTrackRoute(msg.data[0], msg.data[1])
    task_canceled = False
    last_feedback = None
    follow_path_task = RunningTask.NONE
    while not self.navigator.isTaskComplete(task=route_tracking_task):
        feedback = self.navigator.getFeedback(task=route_tracking_task)
        while feedback is not None:
            if not last_feedback or (feedback.last_node_id != last_feedback.last_node_id
or feedback.next_node_id != last_feedback.next_node_id):
                self.get_logger().info('Passed node ' + str(feedback.last_node_id) +
' to next node ' + str(feedback.next_node_id) +
' along edge ' + str(feedback.current_edge_id) + '.')
            last_feedback = feedback
            if feedback.rerouted: # or follow_path_task == RunningTask.NONE
                self.get_logger().info('Passing new route to controller!')
                follow_path_task = self.navigator.followPath(feedback.path)
                feedback = self.navigator.getFeedback(task=route_tracking_task)
            if self.navigator.isTaskComplete(task=follow_path_task):
                self.get_logger().info('Controller or waypoint follower server completed its
task!')
                self.navigator.cancelTask()
                task_canceled = True
        while not self.navigator.isTaskComplete(task=follow_path_task) and not
task_canceled:
            time.sleep(0.1)

    result = self.navigator.getResult()
    if result == TaskResult.SUCCEEDED:
        self.get_logger().info(Fore.GREEN+"navigation task completed."+Fore.RESET)
        self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
    elif result == TaskResult.CANCELED:
        self.get_logger().info(Fore.YELLOW+"navigation task canceled."+Fore.RESET)
    elif result == TaskResult.FAILED:
        self.get_logger().info(Fore.RED+"navigation task failed."+Fore.RESET)
        self.action_feedback_pub.publish(String(data="road_net_nav_failed"))
    else:
        self.get_logger().info(Fore.RED+"navigation task failed."+Fore.RESET)
        self.action_feedback_pub.publish(String(data="road_net_nav_failed"))
```

3.1.2 Arbitrary Pose Navigation

- The initialization function of the `Route_Bridge` class creates a subscriber to the `road_net_nav` topic, receiving `PoseStamped` data (in the format `[start_id, goal_id]`) from the `road_net_nav` topic, and directly calls `getAndTrackRoute` to initiate road network navigation:

```
self.road_net_nav_sub= self.create_subscription(PoseStamped, "road_net_nav",
self.nav_callback, 10)
```

- `PoseStamped`: Message type, containing target location and orientation information.
- `"road_net_nav"`: Topic name, used to send navigation target poses.
- `self.nav_callback`: Callback function, the core logic for handling navigation commands.
- `10`: Message queue size. If callback processing is slow, a maximum of 10 messages will be cached to avoid message loss.
- Step 1: Obtain the robot's current pose (TF transformation): Using `tf2_ros`'s `Buffer` and `TransformListener`, obtain the coordinate transformation from the robot's base coordinate system (`base_footprint`) to the global map coordinate system (`map`), thus obtaining the robot's current pose on the map.
- Step 2: Construct the robot's current pose: Encapsulate the TF transformation result (translation + rotation) into a `start_pose` of type `PoseStamped`, which serves as the **starting pose** for navigation.
- Step 3: Pose to waypoint ID in the road network: This is the **core transformation logic for road network navigation**:
 - Call the `__find_nearest_node` method to match the **starting pose** and **target pose** to the nearest node ID in the road network (using KDTree nearest neighbor search).
 - Call the `getAndTrackRoute` interface of `BasicNavigator`, passing in the starting node ID and the target node ID, to initiate navigation based on the road network nodes.
- Step 4: Final attitude adjustment: Road network navigation only ensures that the robot reaches waypoints near the target pose. The `goToPose` interface allows the robot to precisely adjust to the target pose.

```
def nav_callback(self, msg: PoseStamped):
    ''' 任意位姿路网导航回调函数Arbitrary Pose Road Network Navigation Callback Function'''

    transform = self.tf_buffer.lookup_transform("map", "base_footprint",
rclpy.time.Time(), rclpy.duration.Duration(seconds=5.0))
    if transform is None:
        self.get_logger().info(Fore.RED+"Failed to get TF transform "+Fore.RESET)
        return
    else:
        start_pose = PoseStamped()
        start_pose.header = transform.header
        start_pose.pose.position.x = transform.transform.translation.x
        start_pose.pose.position.y = transform.transform.translation.y
        start_pose.pose.orientation = transform.transform.rotation
```

```

        self.get_logger().info(Fore.CYAN + f"current:
{transform.transform.translation}" + Fore.RESET)

        if self.replace2id:
            nearest_start_node_id, nearest_start_distance =
self.__find_nearest_node(start_pose.pose.position.x, start_pose.pose.position.y)
            nearest_goal_node_id, nearest_goal_distance =
self.__find_nearest_node(msg.pose.position.x, msg.pose.position.y)
            self.get_logger().info(Fore.CYAN + f"nearest_start_node_id:
{nearest_start_node_id}, distance:{nearest_start_distance}"
                                f"\nnearest_goal_node_id:{nearest_goal_node_id},
distance:{nearest_goal_distance}"
                                + Fore.RESET)

            route_tracking_task =
self.navigator.getAndTrackRoute(start=nearest_start_node_id,goal=nearest_goal_node_id)
        else:
            route_tracking_task =
self.navigator.getAndTrackRoute(start=start_pose,goal=msg,use_start=False)
            task_canceled = False
            last_feedback = None
            follow_path_task = RunningTask.NONE
            while not self.navigator.isTaskComplete(task=route_tracking_task):
                feedback = self.navigator.getFeedback(task=route_tracking_task)
                while feedback is not None:
                    if not last_feedback or (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id != last_feedback.next_node_id):
                        self.get_logger().info('Passed node ' + str(feedback.last_node_id) +
' to next node ' + str(feedback.next_node_id) +
' along edge ' + str(feedback.current_edge_id) + '.')
                        last_feedback = feedback
                        if feedback.rerouted: # or follow_path_task == RunningTask.None
                            self.get_logger().info('Passing new route to controller!')
                            follow_path_task = self.navigator.followPath(feedback.path)
                            feedback = self.navigator.getFeedback(task=route_tracking_task)
                        if self.navigator.isTaskComplete(task=follow_path_task):
                            self.get_logger().info('Controller or waypoint follower server completed
its task!')

                            self.navigator.cancelTask()
                            task_canceled = True
                        while not self.navigator.isTaskComplete(task=follow_path_task) and not
task_canceled:
                            time.sleep(0.1)

            result = self.navigator.getResult()
            if result == TaskResult.SUCCEEDED:
                if self.__adjust_pose(msg):
                    self.get_logger().info(Fore.GREEN+"navigation task
completed." + Fore.RESET)
                    self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
            elif result == TaskResult.CANCELED:
                self.get_logger().info(Fore.YELLOW+"navigation task canceled." + Fore.RESET)
            elif result == TaskResult.FAILED:
                self.get_logger().info(Fore.RED+"navigation task failed." + Fore.RESET)

```

```

        self.action_feedback_pub.publish(String(data="road_net_nav_failed"))
    else:
        self.get_logger().info(Fore.RED+"navigation task failed."+Fore.RESET)
        self.action_feedback_pub.publish(String(data="road_net_nav_failed"))

def __adjust_pose(self, final_pose)->bool:
    '''调整到目标姿态Adjust to target posture'''
    self.get_logger().info("adjusting final orientation...")
    final_orientation_task = self.navigator.goToPose(final_pose)
    while not self.navigator.isTaskComplete(task=final_orientation_task):
        time.sleep(0.1)

    orientation_result = self.navigator.getResult()
    if orientation_result == TaskResult.SUCCEEDED:
        return True
    else:
        self.get_logger().info("final orientation adjustment failed.")
        return False

def __find_nearest_node(self, x, y):
    '''使用KDTree查找最近节点Use KDTree to find the nearest node'''
    distance, index = self.tree.query([x, y])
    nearest_node_id = self.node_ids[index]
    return nearest_node_id, distance

```

- The `__load_roadnet_nodes` method is used during the initialization phase of the entire program to parse the **GeoJSON format** road network file, extract `Point` type nodes (topological nodes of the road network), store the coordinates and IDs of the nodes, and build a `KDTree` index.

```

def __load_roadnet_nodes(self):
    '''加载路网节点用于KDTree搜索Load road network nodes for KDTree search'''
    with open(self.road_net_file, 'r') as f:
        data = json.load(f)

    self.node_coords = []    # [[x, y], ...]
    self.node_ids = []      # 对应节点 ID

    for feat in data['features']:
        if feat['geometry']['type'] == 'Point':
            x, y = feat['geometry']['coordinates']
            self.node_coords.append([x, y])
            self.node_ids.append(feat['properties'].get("id", None))

    self.tree = KDTree(self.node_coords)
    self.get_logger().info(f"Loaded {len(self.node_coords)} nodes into KDTree")

```

3.2 Analysis of conver_speed.py

- Because the navigation2 in the ROS2 kilted version publishes speed topics in `TwistStamped` format, while the ROSMASTER-M1 chassis node accepts Twist format speed topics for chassis control, the two cannot communicate directly and require a bridge to convert the speed message format.

We can use the following command to view the difference between the two message types. The main difference is in `std_msgs/Header header`.

- `TwistStamped` format speed topic

```
ros2 interface show geometry_msgs/msg/TwistStamped
```

```
# A twist with reference coordinate frame and timestamp
```

```
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
Twist twist
  Vector3 linear
    float64 x
    float64 y
    float64 z
  Vector3 angular
    float64 x
    float64 y
    float64 z
```

- Twist format speed topic

```
ros2 interface show geometry_msgs/msg/Twist
```

```
# This expresses velocity in free space broken into its linear and angular parts.
```

```
Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
```

- The initialization function of the TwistStampedToTwist class defines a TwistStamped format speed topic subscriber. Then, the callback function converts the TwistStamped format speed topic into a Twist format speed topic and publishes it.

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TwistStamped, Twist
from colorama import Fore
class TwistStampedToTwist(Node):
    def __init__(self):
        super().__init__('conver_speed')
        self.sub = self.create_subscription(
            TwistStamped,
            '/cmd_vel',
            self.callback,
            5
        )
        self.pub = self.create_publisher(
            Twist,
            '/cmd_vel',
            5
        )
        self.get_logger().info(Fore.GREEN + "TwistStamped → Twist The converter has been
started." + Fore.RESET)

    def callback(self, msg: TwistStamped):
        t = Twist()
        t.linear = msg.twist.linear
        t.angular = msg.twist.angular
        self.pub.publish(t)

def main():
    rclpy.init()
    node = TwistStampedToTwist()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

6. Navigation Parameter Debugging

Road network navigation parameter file path:

```
~/ros2_kilted/volumes/code_ws/src/road_net_route/config/nav2_kilted.yaml
```


6.1 Navigation Controller Debugging

Tip

Parameter Adjustment Guide

- If you need to adjust the movement speed and rotation angular velocity during navigation, adjust `desired_linear_vel` and `max_angular_accel`
- If you need to adjust the accuracy of reaching the target point, adjust `xy_goal_tolerance` and `yaw_goal_tolerance`
- For other parameter adjustments, see the comments below.

```
controller_server:
  ros__parameters:
    # ===== Controller Core Global Parameters =====
    # Controller's main loop frequency (Hz), determines the frequency of local path planning
    and speed command issuance, Must match the hardware execution frequency.
    controller_frequency: 20.0
    # Cost map update timeout (seconds) If the cost map is not updated within this time, the
    controller stops outputting speed commands to avoid planning based on an outdated map.
    costmap_update_timeout: 0.30
    # Minimum speed threshold (m/s) in the X direction (forward/backward). Speed commands
    below this value are considered invalid filter minor jitter.
    min_x_velocity_threshold: 0.001
    # Minimum speed threshold (m/s) in the Y direction (lateral movement). Only applicable
    to omnidirectional mobile robots (such as Mecanum wheels). Differential drive robots can be
    set to a larger value (such as 0.5) to disable lateral movement.
    min_y_velocity_threshold: 0.5
    # Minimum speed threshold for rotation direction (radians/second). Rotation commands
    below this value are considered invalid, filtering out minor rotational jitter.
    min_theta_velocity_threshold: 0.001
    # Controller failure tolerance (meters). When the deviation of the locally planned path
    exceeds this value, the controller is considered to have failed, triggering replanning.
    failure_tolerance: 0.3
    # List of progress checker plugins, used to monitor whether the robot is moving towards
    the target.
    progress_checker_plugins: ["progress_checker"]
    # List of goal checker plugins, used to determine whether the robot has reached the
    target pose.
    goal_checker_plugins: ["general_goal_checker"]
    # List of controller plugins, the core of which is the local path tracking algorithm.
    Here, the pure tracking controller corresponding to FollowPath is used.
    controller_plugins: ["FollowPath"]
    # whether to use real-time priority scheduling of controller threads.
    use_realtime_priority: false
    # Speed ••limit topic name. The controller will subscribe to speed commands for this
    topic and dynamically adjust the maximum linear/angular velocity (e.g., slowing down during
    obstacle avoidance).
    speed_limit_topic: "speed_limit"

    # ===== Progress Checker Parameters (SimpleProgressChecker)
    =====
```

```

progress_checker:
    # Progress checker plugin type. SimpleProgressChecker is the basic implementation,
    monitoring the robot's movement distance and time.
    plugin: "nav2_controller::SimpleProgressChecker"
    # The minimum distance (in meters) the robot needs to move. If it doesn't move this
    distance within the movement_time_allowance time, it's considered no progress.
    required_movement_radius: 0.05
    # The maximum allowed no progress time (in seconds). If required_movement_radius is
    not met within this time, the controller will abort the task (error code 105).
    movement_time_allowance: 10.0

# ===== Target Checker Parameters (SimpleGoalChecker)
=====
general_goal_checker:
    # Whether it is a stateful checker. Setting it to True will continuously monitor the
    target pose until the tolerance is met; False will only perform a single check.
    stateful: True
    # Target checker plugin type. SimpleGoalChecker is the basic implementation,
    determining whether the position and angle are within the tolerance range.
    plugin: "nav2_controller::SimpleGoalChecker"
    # XY plane position tolerance (meters). When the deviation between the robot's
    position and the target position is less than this value, the position is considered
    reached.
    xy_goal_tolerance: 0.15
    # Yaw angle tolerance (radians). When the deviation between the robot's orientation
    and the target orientation is less than this value, the angle is considered reached.
    yaw_goal_tolerance: 0.20

# ===== Pure Tracking Controller Parameters
(RegulatedPurePursuitController) =====
FollowPath:
    # Local path tracking plugin type. RegulatedPurePursuitController is a pure tracking
    algorithm with speed adjustment, suitable for differential drive robots.
    plugin: "nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
    # Desired linear velocity (m/s): The target linear velocity of the robot during normal
    movement, dynamically scaled by obstacle avoidance/curvature adjustment.
    desired_linear_vel: 0.16
    # Lookahead distance (m): A core parameter of the pure tracking algorithm, determining
    the distance between path points tracked by the robot. It must match the robot's speed.
    lookahead_dist: 0.5
    # Minimum lookahead distance (m): Prevents the robot from frequently turning due to
    excessively small lookahead distance.
    min_lookahead_dist: 0.05
    # Maximum lookahead distance (meters), preventing excessively large lookahead
    distances from causing sluggish robot responses.
    max_lookahead_dist: 0.9
    # Lookahead time (seconds). If use_velocity_scaled_lookahead_dist is enabled, lookahead
    distance = desired_linear_vel * lookahead_time
    lookahead_time: 1.5
    # Angular velocity (radians/second) for rotating to the target orientation. The
    rotational speed used to adjust the orientation after the robot reaches the position.
    rotate_to_heading_angular_vel: 1.3

```

```
# Transformer tolerance (seconds). The maximum deviation between the robot's pose
transformation timestamp and the current time, addressing Transformer latency issues.
transform_tolerance: 0.5

# Whether to use velocity-scaled lookahead distance. When set to True, the lookahead
distance dynamically changes with the robot's speed (the faster the speed, the farther the
lookahead).
use_velocity_scaled_lookahead_dist: false

# Minimum linear velocity (m/s) when approaching the target. When the robot is close
to the target, it slows down to this value to avoid overshooting it.
min_approach_linear_velocity: 0.05

# Approach velocity scaling distance (m). When the robot is less than this value from
the target, it begins to linearly slow down (from desired_linear_vel to
min_approach_linear_velocity).
approach_velocity_scaling_dist: 1.0

# Whether to enable collision detection. When set to True, the controller will check
for collisions on the look-ahead path to avoid obstacles.
use_collision_detection: false

# Maximum allowed collision time (seconds) to the look-ahead point. Only effective
when use_collision_detection is True, used to assess collision risk.
max_allowed_time_to_collision_up_to_carrot: 1.0

# Whether to enable linear velocity adjustment based on path curvature. When set to
True, the smaller the turning radius, the lower the linear velocity.
use_regulated_linear_velocity_scaling: true

# Whether to enable linear velocity scaling based on cost map. When set to True, the
closer the obstacle, the lower the linear velocity.
use_cost_regulated_linear_velocity_scaling: false

# Minimum turning radius (meters) to trigger curvature scaling. When the turning
radius corresponding to the path curvature is less than this value, deceleration begins.
regulated_linear_scaling_min_radius: 0.05

# Minimum speed (meters/second) for curvature scaling. Even with a very small turning
radius, the linear velocity will not fall below this value.
regulated_linear_scaling_min_speed: 0.25

# Whether to use fixed curvature lookahead distance. When set to True, the lookahead
distance is determined by curvature_lookahead_dist.
use_fixed_curvature_lookahead: false

# Curvature lookahead distance (meters). Only effective when
use_fixed_curvature_lookahead is True.
curvature_lookahead_dist: 1.0

# Enable "Rotate to Heading Before Moving". When set to True, if the robot's heading
deviates significantly from the path direction, it will rotate before moving forward.
use_rotate_to_heading: true

# Minimum angle (radians) required to trigger rotation to heading. Rotation begins
when the robot's heading deviates from the path direction beyond this value.
rotate_to_heading_min_angle: 0.785 #Approximately 45 degrees

# Maximum angular acceleration (radians/second2). Limits the robot's rotational
acceleration to prevent sudden turns that could damage hardware.
max_angular_accel: 2.7

# Maximum distance (meters) for robot pose search. Searches for path points within
this distance when the controller cannot find a matching robot pose on the path.
max_robot_pose_search_dist: 10.0

# Whether to interpolate curvature after the target point. When set to True, curvature
is interpolated on the path after the target point to prevent the robot from suddenly
stopping after reaching the target.
```

```

    interpolate_curvature_after_goal: false
    # Cost scaling distance (meters), only effective when
    use_cost_regulated_linear_velocity_scaling is True, used to calculate the weight of obstacle
    avoidance deceleration.
    cost_scaling_dist: 0.08
    # cost scaling gain, only effective when use_cost_regulated_linear_velocity_scaling is
    True. The larger the gain, the more significant the obstacle avoidance deceleration.
    cost_scaling_gain: 1.0
    # inflation cost scaling factor, used to convert the inflation layer cost of the cost
    map into a velocity scaling factor. The larger the value, the more sensitive it is to
    obstacle avoidance in the inflation region.
    inflation_cost_scaling_factor: 3.0

```

6.2 Obstacle Avoidance and Cost Map Debugging

💡 Tip

- If the robot gets too close to obstacles on the map during navigation, you can adjust the inflation region to move the robot away from the obstacles. Increase the inflation radius and cost scaling factor in `local_costmap` and `global_costmap`. The larger the value, the less likely the robot is to get too close to obstacles.
- See the comments below for other parameter adjustments.

```

# Local Cost Map Configuration: Short-term cost map for real-time obstacle avoidance,
# updated by scrolling as the robot moves
local_costmap:
  local_costmap:
    ros__parameters:
      # ===== Core Global Parameters of the Cost Map =====
      # Cost map update frequency (Hz), determines the refresh speed of the local map, must
      match the sensor frequency.
      update_frequency: 5.0
      # Cost map publication frequency (Hz), the frequency of publishing the map to
      visualization tools such as RViz; a frequency lower than the update frequency can reduce
      bandwidth usage.
      publish_frequency: 2.0
      # Global coordinate system of the cost map; the local map uses the odom coordinate
      system.
      global_frame: odom
      # Robot base coordinate system, consistent with the robot's center coordinate system
      configured in the hardware.
      robot_base_frame: base_link
      # Whether to enable scrolling window mode, a core feature of the local map: the map
      scrolls with the robot's movement, only retaining the map of the area around the robot.
      rolling_window: true
      # Scroll window width (meters), the horizontal range of the local map
      width: 3
      # Scroll window height (meters) - vertical range of local map
      height: 3

```

```

    # Cost map resolution (meters/pixel), representing the actual distance represented by
    each pixel, 0.05 is 5 centimeters/pixel
    resolution: 0.05

    # Robot outline (polygon), representing the actual shape of the robot using a
    coordinate list
    footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"

    # Robot outline padding distance (meters), an extra safety distance extended outside
    the actual outline for obstacle avoidance buffering
    footprint_padding: 0.0616

    # Cost map layer plugin list, loaded in order (voxel_layer: voxel obstacle layer;
    inflation_layer: inflation layer)
    plugins: ["voxel_layer", "inflation_layer"]

    # List of cost map filter plugins used for post-processing the map (keepout_filter:
    restricted area filtering)
    filters: ["keepout_filter"]

    # Whether to always publish the full cost map. Setting it to True publishes the full
    map on every update, while False only publishes changed areas (reducing bandwidth)
    always_send_full_costmap: True

    # Service introspection mode. Disabled disables service introspection; options are
    enabled/verbose
    service_introspection_mode: "disabled"

    # ===== Filter Plugin: Restricted Area Filter (KeepoutFilter)
    =====
    keepout_filter:
        # Filter plugin type. KeepoutFilter is used to mark restricted areas in the cost map
        plugin: "nav2_costmap_2d::KeepoutFilter"
        # Enables restricted area filtering. `KEEPOUT_ZONE_ENABLED` is an external parameter
        (must be set to true/false at startup).
        enabled: KEEPOUT_ZONE_ENABLED
        # Subscribes to restricted area information, receiving the geometry and location
        information of the restricted areas (e.g., `PolygonStamped`)
        filter_info_topic: "keepout_costmap_filter_info"
        # Whether to override the lethal cost of the restricted areas. Setting it to True
        replaces the default lethal cost (255) with a custom cost.
        override_lethal_cost: True
        # Custom lethal cost of the restricted areas. 200 indicates a lower lethal cost
        (255) than the default, but still a high-cost restricted area.
        lethal_override_cost: 200

    # ===== Layer Plugin: InflationLayer =====
    inflation_layer:
        # Layer plugin type, InflationLayer, used to generate an inflation zone around
        obstacles to avoid robot collisions.
        plugin: "nav2_costmap_2d::InflationLayer"
        # Cost scaling factor, controls the cost decay rate of the inflation zone (the
        larger the value, the faster the cost decays and the smaller the inflation zone)
        cost_scaling_factor: 3.0
        # Inflation radius (meters), the inflation range around the obstacle, the minimum
        safe distance from the robot outline to the obstacle
        inflation_radius: 0.1

    # Global cost map configuration: a long-term cost map used for global path planning
    of the robot, covering the entire environment

```

```

global_costmap:
  global_costmap:
    ros__parameters:
      # ===== Core Global Parameters of the Cost Map =====
      # Cost map update frequency (Hz), the global map does not need high-frequency updates,
      set to 1-2Hz is sufficient
      update_frequency: 1.0
      # Cost map publication frequency (Hz): Consistent with update frequency
      publish_frequency: 1.0
      # Global coordinate system of the cost map (map=globally consistent for path planning)
      global_frame: map
        # Robot base coordinate system (consistent with local map)

      # robot_radius: 0.22

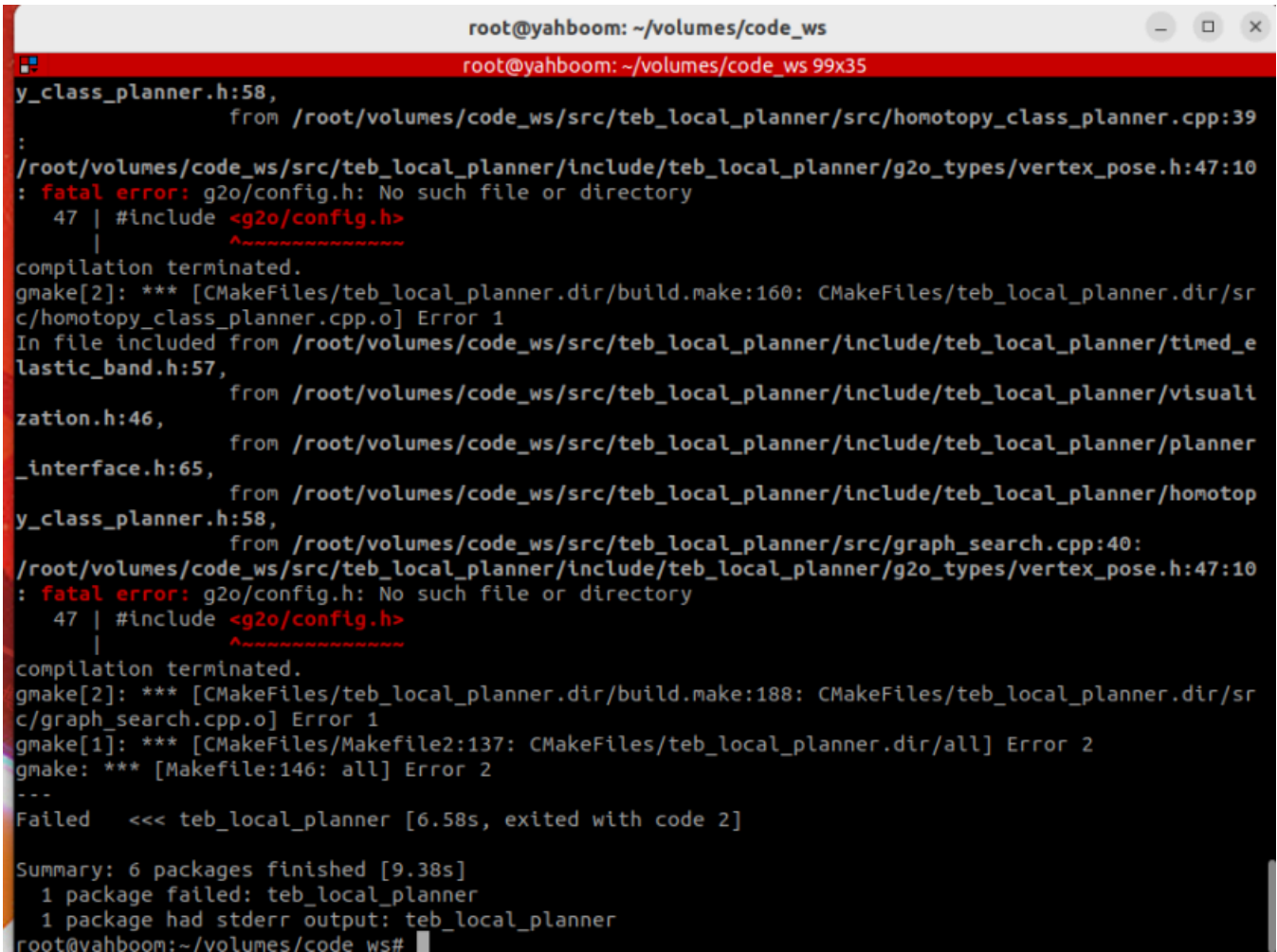
      robot_base_frame: base_link
      # Robot radius (replaced by footprint, commented out)
      # robot_radius: 0.22
      # Robot footprint (consistent with local map)
      footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"
      # Robot outline padding distance (consistent with the local map)
      footprint_padding: 0.0616
      # Cost map resolution (consistent with the local map, ensuring coordinate matching)
      resolution: 0.05
      # Whether to track unknown space. Setting it to True marks unexplored areas on the map
      (used for SLAM and unknown environment planning).
      track_unknown_space: true
      # List of cost map layer plugins (static_layer: static layer; obstacle_layer: 2D
      obstacle layer; inflation_layer: inflation layer)
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      # List of cost map filter plugins (keepout_filter: no-go zone filter; speed_filter:
      speed limit filter)
      filters: ["keepout_filter", "speed_filter"]
      # Whether to always publish the full cost map
      always_send_full_costmap: True
      # Service introspection mode
      service_introspection_mode: "disabled"

      # ===== Layer Plugin: InflationLayer =====
      inflation_layer:
        # Layer plugin type. InflationLayer is used to generate inflation zones around
        obstacles to avoid robot collisions.
        plugin: "nav2_costmap_2d::InflationLayer"
        # Cost scaling factor controls the rate of cost decay in the inflation region (the
        larger the value, the faster the cost decays and the smaller the inflation region).
        cost_scaling_factor: 3.0
        # Inflation radius (meters): The inflation range around the obstacle; the minimum
        safe distance from the robot's outline to the obstacle.
        inflation_radius: 0.1

```

7. Notes

- If you encounter an error during compilation of a functional package within the road network navigation Docker container, specifically an error indicating that the g2o header file cannot be found, you can resolve this by following these steps:

A terminal window titled 'root@yahboom: ~/volumes/code_ws' showing the output of a catkin build. The output displays two 'fatal error' messages: 'g2o/config.h: No such file or directory' at line 47 of 'g2o/config.h' in 'homotopy_class_planner.cpp' and 'graph_search.cpp'. The build fails with 'Error 2' for the 'teb_local_planner' package. The terminal text is as follows:

```
root@yahboom: ~/volumes/code_ws
root@yahboom: ~/volumes/code_ws 99x35
y_class_planner.h:58,
    from /root/volumes/code_ws/src/teb_local_planner/src/homotopy_class_planner.cpp:39
:
/root/volumes/code_ws/src/teb_local_planner/include/teb_local_planner/g2o_types/vertex_pose.h:47:10
: fatal error: g2o/config.h: No such file or directory
 47 | #include <g2o/config.h>
    |
compilation terminated.
gmake[2]: *** [CMakeFiles/teb_local_planner.dir/build.make:160: CMakeFiles/teb_local_planner.dir/src/homotopy_class_planner.cpp.o] Error 1
In file included from /root/volumes/code_ws/src/teb_local_planner/include/teb_local_planner/timed_elastic_band.h:57,
    from /root/volumes/code_ws/src/teb_local_planner/include/teb_local_planner/visualization.h:46,
    from /root/volumes/code_ws/src/teb_local_planner/include/teb_local_planner/planner_interface.h:65,
    from /root/volumes/code_ws/src/teb_local_planner/include/teb_local_planner/homotopy_class_planner.h:58,
    from /root/volumes/code_ws/src/teb_local_planner/src/graph_search.cpp:40:
/root/volumes/code_ws/src/teb_local_planner/include/teb_local_planner/g2o_types/vertex_pose.h:47:10
: fatal error: g2o/config.h: No such file or directory
 47 | #include <g2o/config.h>
    |
compilation terminated.
gmake[2]: *** [CMakeFiles/teb_local_planner.dir/build.make:188: CMakeFiles/teb_local_planner.dir/src/graph_search.cpp.o] Error 1
gmake[1]: *** [CMakeFiles/Makefile2:137: CMakeFiles/teb_local_planner.dir/all] Error 2
gmake: *** [Makefile:146: all] Error 2
---
Failed <<< teb_local_planner [6.58s, exited with code 2]

Summary: 6 packages finished [9.38s]
 1 package failed: teb_local_planner
 1 package had stderr output: teb_local_planner
root@yahboom: ~/volumes/code_ws#
```

Solution:

```
cd ~/volumes/g2o/build/
sudo make install
```

After successful execution, compile the functional package again:

```
cd ~/volumes/code_ws/
colcon build
```



```

root@yahboom:~/volumes/g2o/build# cd ~/volumes/code_ws/
root@yahboom:~/volumes/code_ws# colcon build
Starting >>> costmap_converter_msgs
Starting >>> road_net_route
Starting >>> yahboomcar_description
Starting >>> yahboomcar_interfaces
Finished <<< costmap_converter_msgs [1.20s]
Starting >>> costmap_converter
Starting >>> teb_msgs
Finished <<< costmap_converter [0.55s]
Finished <<< yahboomcar_interfaces [2.17s]
Finished <<< teb_msgs [1.28s]
Starting >>> teb_local_planner
Finished <<< teb_local_planner [0.31s]
Finished <<< yahboomcar_description [3.63s]
Finished <<< road_net_route [3.72s]

Summary: 7 packages finished [3.95s]
root@yahboom:~/volumes/code_ws#

```

- If a .xml behavior tree error occurs during navigation:

```

root@yahboom: ~
root@yahboom: ~ 114x35
[component_container_isolated-1] [INFO] [1766458256.918716249] [velocity_smoother]: Activating
[component_container_isolated-1] [INFO] [1766458256.918879041] [velocity_smoother]: Creating bond (velocity_smoother) to lifecycle manager.
[component_container_isolated-1] [INFO] [1766458257.029635352] [lifecycle_manager_navigation]: Server velocity_smoother connected with bond.
[component_container_isolated-1] [INFO] [1766458257.029735133] [lifecycle_manager_navigation]: Activating collision_monitor
[component_container_isolated-1] [INFO] [1766458257.030314553] [collision_monitor]: Activating
[component_container_isolated-1] [INFO] [1766458257.030446623] [collision_monitor]: Creating bond (collision_monitor) to lifecycle manager.
[component_container_isolated-1] [INFO] [1766458257.135733966] [lifecycle_manager_navigation]: Server collision_monitor connected with bond.
[component_container_isolated-1] [INFO] [1766458257.135845395] [lifecycle_manager_navigation]: Activating bt_navigator
[component_container_isolated-1] [INFO] [1766458257.136481426] [bt_navigator]: Activating
[component_container_isolated-1] [WARNING] [1766458257.136481426] [bt_navigator]: You executed BehaviorTreeFactory::createTreeFromFile after registerBehaviorTreeFrom[File/Text].
[component_container_isolated-1] This is NOT, probably, what you want to do.
[component_container_isolated-1] You should probably use BehaviorTreeFactory::createTree, instead
[component_container_isolated-1] [ERROR] [1766458258.195179511] [bt_navigator_navigate_to_pose_rclcpp_node]: "spin" action server not available after waiting for 1.00s
[component_container_isolated-1] [ERROR] [1766458258.247135947] [bt_navigator]: Setting internal error error_code: 9001, error_msg:Exception when creating BT tree from file: Action server spin not available
[component_container_isolated-1] [ERROR] [1766458258.247426137] [bt_navigator]: Error loading XML file: /opt/ros/kilted/share/nav2_bt_navigator/behavior_trees/navigate_to_pose_w_replanning_and_recovery.xml
[component_container_isolated-1] [INFO] [1766458258.247471868] [bt_navigator]: Deactivating
[component_container_isolated-1] [INFO] [1766458258.247532254] [bt_navigator]: Destroying bond (bt_navigator) to lifecycle manager.
[component_container_isolated-1] [ERROR] [1766458258.249267089] [lifecycle_manager_navigation]: Failed to change state for node: bt_navigator
[component_container_isolated-1] [ERROR] [1766458258.249528734] [lifecycle_manager_navigation]: Failed to bring up all requested nodes. Aborting bringup.
[component_container_isolated-1] [INFO] [1766458260.574333358] [global_costmap.global_costmap]: StaticLayer: Resizing costmap to 328 X 343 at 0.050000 m/pix

```

Answer: Modify the two behavior tree files by deleting two lines from the file content.

In the road network (Docker) terminal, enter:

```

sudo nano
/opt/ros/kilted/share/nav2_bt_navigator/behavior_trees/navigate_to_pose_w_replanning_and_recovery.xml

sudo gedit
/opt/ros/kilted/share/nav2_bt_navigator/behavior_trees/navigate_through_poses_w_replanning_and_recovery.xml

```



```
root@yahboom: ~  
root@yahboom: ~ 114x35  
/opt/ros/kilted/share/nav2_bt_navigator/behavior_trees/navigate_to_pose_w_replanning_and_recovery.xml  
</RateController>  
<RecoveryNode number_of_retries="1" name="FollowPath">  
  <FollowPath path="{path}" controller_id="{selected_controller}" error_code_id="{follow_path_error_code}">  
    <Sequence>  
      <WouldAControllerRecoveryHelp error_code="{follow_path_error_code}" />  
      <ClearEntireCostmap name="ClearLocalCostmap-Context" service_name="local_costmap/clear_entirely_local" />  
    </Sequence>  
  </FollowPath>  
</RecoveryNode>  
</PipelineSequence>  
<Sequence>  
  <Fallback>  
    <WouldAControllerRecoveryHelp error_code="{follow_path_error_code}" />  
    <WouldAPlannerRecoveryHelp error_code="{compute_path_error_code}" />  
  </Fallback>  
<ReactiveFallback name="RecoveryFallback">  
  <GoalUpdated />  
  <RoundRobin name="RecoveryActions">  
    <Sequence name="ClearingActions">  
      <ClearEntireCostmap name="ClearLocalCostmap-Subtree" service_name="local_costmap/clear_entirely_local" />  
      <ClearEntireCostmap name="ClearGlobalCostmap-Subtree" service_name="global_costmap/clear_entirely_global" />  
    </Sequence>  
    <Spin spin_dist="1.57" error_code_id="{spin_error_code}" error_msg="{spin_error_msg}" />  
    <Wait wait_duration="5.0" error_code_id="{wait_error_code}" error_msg="{wait_error_msg}" />  
    <Backup backup_dist="0.30" backup_speed="0.15" error_code_id="{backup_error_code}" error_msg="{backup_error_msg}" />  
  </RoundRobin>  
</ReactiveFallback>  
</Sequence>  
</RecoveryNode>  
</BehaviorTree>  
</root>  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

After deleting the lines, press Ctrl+X, then press Y and Enter to save and exit.